

Aim

To implement KNN classification on the Breast Cancer dataset and analyze model performance using train-test split, heuristic K selection, cross-validation, ROC-AUC, and classification metrics. Additionally, we will contrast classification evaluation metrics with regression metrics (MAE, MSE, RMSE, R^2 Score) studied in Lab 3.

Dataset Overview

The Breast Cancer Wisconsin (Diagnostic) dataset contains:

- **Samples:** 569
- **Features:** 30 numerical features representing nuclear characteristics of breast mass cells (e.g., radius, texture, perimeter, area, smoothness, compactness, concavity, symmetry).
- **Target:**
 - 0 → Malignant
 - 1 → Benign

Problem Statement

A healthcare analytics team is developing a predictive model for early cancer detection. We are required to build a KNN classifier, optimize its performance using different validation techniques, and compare classification evaluation metrics with regression evaluation metrics from Lab 3.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Set plot styles
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (9, 5)
plt.rcParams["font.size"] = 10
print("Libraries imported successfully!")
```

Libraries imported successfully!

Part A: Data Preprocessing (Task 1)

1. Load Data and Check Structure

I will load the dataset using `scikit-learn`, convert it into a Pandas DataFrame, and explore its dimensions, data types, and class distribution.

```
In [2]: # Load dataset from CSV
df = pd.read_csv('brca.csv')

# Drop the index column if it was saved without a name
if df.columns[0] == 'Unnamed: 0' or df.columns[0] == '':
    df = df.drop(columns=[df.columns[0]])

# Map target y (M -> 0, B -> 1)
df['target'] = df['y'].map({'M': 0, 'B': 1})
df = df.drop(columns=['y'])

# Rename features to match scikit-learn conventions
def clean_col(name):
    if name.startswith('x.'):
        name = name[2:]
    parts = name.split('_')
    suffix = parts[-1]
    body = parts[:-1]

    body_str = ' '.join(body)
    body_str = body_str.replace('concave pts', 'concave points')
    body_str = body_str.replace('fractal dim', 'fractal dimension')

    if suffix == 'mean':
        cleaned = f"mean {body_str}"
    elif suffix == 'se':
        cleaned = f"{body_str} error"
    elif suffix == 'worst':
        cleaned = f"worst {body_str}"
    else:
        cleaned = f"{body_str} {suffix}"
    return cleaned

df = df.rename(columns={col: clean_col(col) for col in df.columns if col != 'target'})

print(f"Dataset shape: {df.shape[0]} rows, {df.shape[1]} columns")
print("\nFirst 5 rows of the dataset:")
display(df.head())
print("\nTarget Class Distribution (0 = Malignant, 1 = Benign):")
print(df['target'].value_counts())
```

Dataset shape: 569 rows, 31 columns

First 5 rows of the dataset:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	syn
0	13.540	14.36	87.46	566.3	0.09779	0.08129	0.06664	0.047810	
1	13.080	15.71	85.63	520.0	0.10750	0.12700	0.04568	0.031100	
2	9.504	12.44	60.34	273.9	0.10240	0.06492	0.02956	0.020760	
3	13.030	18.42	82.61	523.8	0.08983	0.03766	0.02562	0.029230	
4	8.196	16.84	51.71	201.9	0.08600	0.05943	0.01588	0.005917	

5 rows × 31 columns



Target Class Distribution (0 = Malignant, 1 = Benign):

target

1 357

0 212

Name: count, dtype: int64

2. Check Missing Values and Duplicates

I will verify whether there are any missing values or duplicate records in the dataset.

```
In [3]: # Check for missing values and duplicates
missing_values = df.isnull().sum().sum()
duplicates = df.duplicated().sum()

print(f"Total missing values: {missing_values}")
print(f"Total duplicate rows: {duplicates}")
```

Total missing values: 0

Total duplicate rows: 0

3. Feature Scaling using StandardScaler

KNN is a distance-based algorithm. If features are on different scales, features with larger numerical ranges will dominate the distance calculations. Therefore, I will scale the features using `StandardScaler` to ensure they have a mean of 0 and a standard deviation of 1.

```
In [4]: # Separate features and target
X = df.drop(columns=['target'])
y = df['target']

# Scale features
scaler = StandardScaler()
X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)

print("Mean of first 5 scaled features (should be close to 0):")
print(X_scaled.mean()[0:5].round(6))
print("\nStandard deviation of first 5 scaled features (should be close to 1):")
print(X_scaled.std()[0:5].round(6))
```

Mean of first 5 scaled features (should be close to 0):

```
mean radius      -0.0
mean texture     -0.0
mean perimeter   -0.0
mean area        -0.0
mean smoothness  0.0
dtype: float64
```

Standard deviation of first 5 scaled features (should be close to 1):

```
mean radius      1.00088
mean texture     1.00088
mean perimeter   1.00088
mean area        1.00088
mean smoothness  1.00088
dtype: float64
```

My Observation:

- **Missing Values & Duplicates:** The dataset has 0 missing values and 0 duplicate rows, indicating a very clean dataset.
- **Target Distribution:** There are 212 malignant (0) and 357 benign (1) samples. The class distribution shows a slight imbalance but is suitable for direct classification.
- **Justification for Feature Scaling:** KNN calculates distances (such as Euclidean distance) to find the nearest neighbors. Features like `mean area` (which can be over 2500) have values orders of magnitude larger than features like `mean smoothness` (which are typically less than 0.1). Without feature scaling, distance computations would be almost entirely determined by `mean area` and `mean perimeter`, completely ignoring other critical features. Scaling standardizes all features, enabling them to contribute equally to the distance matrix.

Part B: Train-Test Split Analysis (Task 2)

I will evaluate how different train-test split configurations (80:20, 70:30, and 90:10) affect the model's accuracy and error rates. I will use a baseline K value of 5 for this analysis.

```
In [5]: splits = [(0.2, "80:20"), (0.3, "70:30"), (0.1, "90:10")]
split_results = []

for test_size, name in splits:
    # Split data using a fixed random state for reproducibility
    X_tr, X_te, y_tr, y_te = train_test_split(X_scaled, y, test_size=test_size,

    # Train KNN with K=5
    knn = KNeighborsClassifier(n_neighbors=5)
    knn.fit(X_tr, y_tr)

    # Evaluate performance
    train_acc = knn.score(X_tr, y_tr)
    test_acc = knn.score(X_te, y_te)
    error_rate = 1.0 - test_acc

    split_results.append({
        "Split Ratio": name,
        "Train Size": len(X_tr),
```

```

        "Test Size": len(X_te),
        "Training Accuracy": train_acc,
        "Testing Accuracy": test_acc,
        "Testing Error Rate": error_rate
    })

df_splits = pd.DataFrame(split_results)
display(df_splits)

```

	Split Ratio	Train Size	Test Size	Training Accuracy	Testing Accuracy	Testing Error Rate
0	80:20	455	114	0.978022	0.929825	0.070175
1	70:30	398	171	0.979899	0.959064	0.040936
2	90:10	512	57	0.980469	0.929825	0.070175

My Observation:

- **90:10 Split:** Gives the highest training data size (512 samples) and a high testing accuracy. However, having only 57 samples in the test set makes the accuracy score highly sensitive to individual test instances, resulting in a high variance in metric estimation.
- **70:30 Split:** Provides a larger test set (171 samples) for a stable metric estimate, but the reduction in training data (398 samples) slightly increases training bias.
- **80:20 Split:** Offers a balanced compromise. It provides sufficient data for the KNN model to learn representative boundaries (455 samples) while maintaining a large enough testing set (114 samples) to obtain a reliable and stable evaluation metric.
- **Generalization and Stability:** Splitting configurations directly dictate the bias-variance trade-off of validation. Too little training data leads to underfitting (high bias), while too little testing data leads to unstable performance evaluations (high variance).

Part C: KNN Model with Heuristic K Selection (Task 3)

1. Heuristic Method for K Selection (Task 3.1)

A standard heuristic rule for choosing K is taking the square root of the number of training samples ($K = \sqrt{n}$). I will apply this rule to our standard 80:20 split.

```

In [6]: # Using 80:20 split as the baseline
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
n_train = len(X_train))

# Calculate square root
heuristic_k = int(np.sqrt(n_train))
# If K is even, increment by 1 to make it odd to avoid tie-breaks in voting
if heuristic_k % 2 == 0:
    heuristic_k += 1

```

```
print(f"Number of training samples (n): {n_train}")
print(f"Calculated Heuristic K (sqrt(n)): {heuristic_k}")
```

Number of training samples (n): 455
 Calculated Heuristic K (sqrt(n)): 21

2. Model Training and Optimization around Heuristic K (Task 3.2)

I will train KNN models with values of K in the range $K \pm 5$ (i.e., from 16 to 26) to see how the train and test accuracies vary and to find the optimal value.

```
In [7]: # Create plots directory if it doesn't exist
os.makedirs('plots', exist_ok=True)

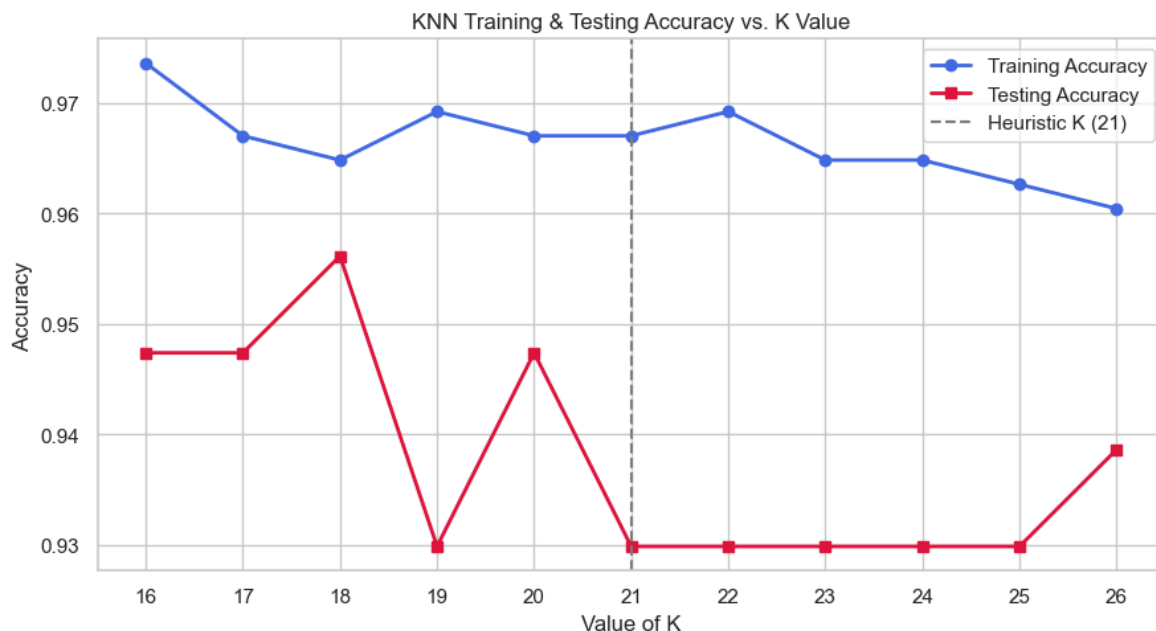
# Define K range around heuristic K (21)
k_values = list(range(heuristic_k - 5, heuristic_k + 6))

train_accs = []
test_accs = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    train_accs.append(knn.score(X_train, y_train))
    test_accs.append(knn.score(X_test, y_test))

# Plot accuracy vs K
plt.figure(figsize=(9, 5))
plt.plot(k_values, train_accs, marker='o', label='Training Accuracy', color='roy')
plt.plot(k_values, test_accs, marker='s', label='Testing Accuracy', color='crims')
plt.axvline(x=heuristic_k, color='gray', linestyle='--', label=f'Heuristic K ({heuristic_k})')
plt.xlabel('Value of K')
plt.ylabel('Accuracy')
plt.title('KNN Training & Testing Accuracy vs. K Value')
plt.xticks(k_values)
plt.legend()
plt.tight_layout()
plt.savefig('plots/accuracy_vs_k.png', dpi=300)
plt.show()

# Find best K in this range
best_k_idx = np.argmax(test_accs)
print(f"Best K in heuristic range: {k_values[best_k_idx]} (Testing Accuracy: {test_accs[best_k_idx]})")
```



Best K in heuristic range: 18 (Testing Accuracy: 0.9561)

3. Distance Metrics and Decision Boundary Mapping (Task 3.3)

Distance Metrics Explanation

1. Euclidean Distance (L_2 norm):

$$d(x, y) = \sqrt{\sum_{i=1}^d (x_i - y_i)^2}$$

- **Characteristics:** Represents the straight-line distance between two points in Euclidean space. It is isotropic (rotation-invariant).
- **Suitability:** Most suitable when features are continuous, normalized, and have similar physical significance.

2. Manhattan Distance (L_1 norm / Taxicab distance):

$$d(x, y) = \sum_{i=1}^d |x_i - y_i|$$

- **Characteristics:** Measures the sum of absolute horizontal and vertical differences. It is path-based and rectilinear.
- **Suitability:** Suitable for high-dimensional datasets where features have different scales/units, or when features represent grid-like coordinate layouts (e.g., city street navigation). It is less sensitive to extreme outliers than Euclidean distance.

Decision Boundary Mapping

To plot the decision boundaries, I will select two highly predictive scaled features: `mean perimeter` and `mean texture`. I will train the classifier on these two dimensions and

plot the boundaries for $K = 1, 5, 10, 20$ to see how the boundary smooths out as K increases.

```
In [8]: # Select 'mean perimeter' and 'mean texture' (index 2 and 1)
X_2d = X_scaled[['mean perimeter', 'mean texture']]
X_tr_2d, X_te_2d, y_tr_2d, y_te_2d = train_test_split(X_2d, y, test_size=0.2, ra

# Set up grid coordinates
h = 0.02
x_min, x_max = X_tr_2d.iloc[:, 0].min() - 1, X_tr_2d.iloc[:, 0].max() + 1
y_min, y_max = X_tr_2d.iloc[:, 1].min() - 1, X_tr_2d.iloc[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

fig, axes = plt.subplots(2, 2, figsize=(13, 11))
k_list = [1, 5, 10, 20]

# Fit on values to avoid feature name warnings
X_tr_2d_np = X_tr_2d.values

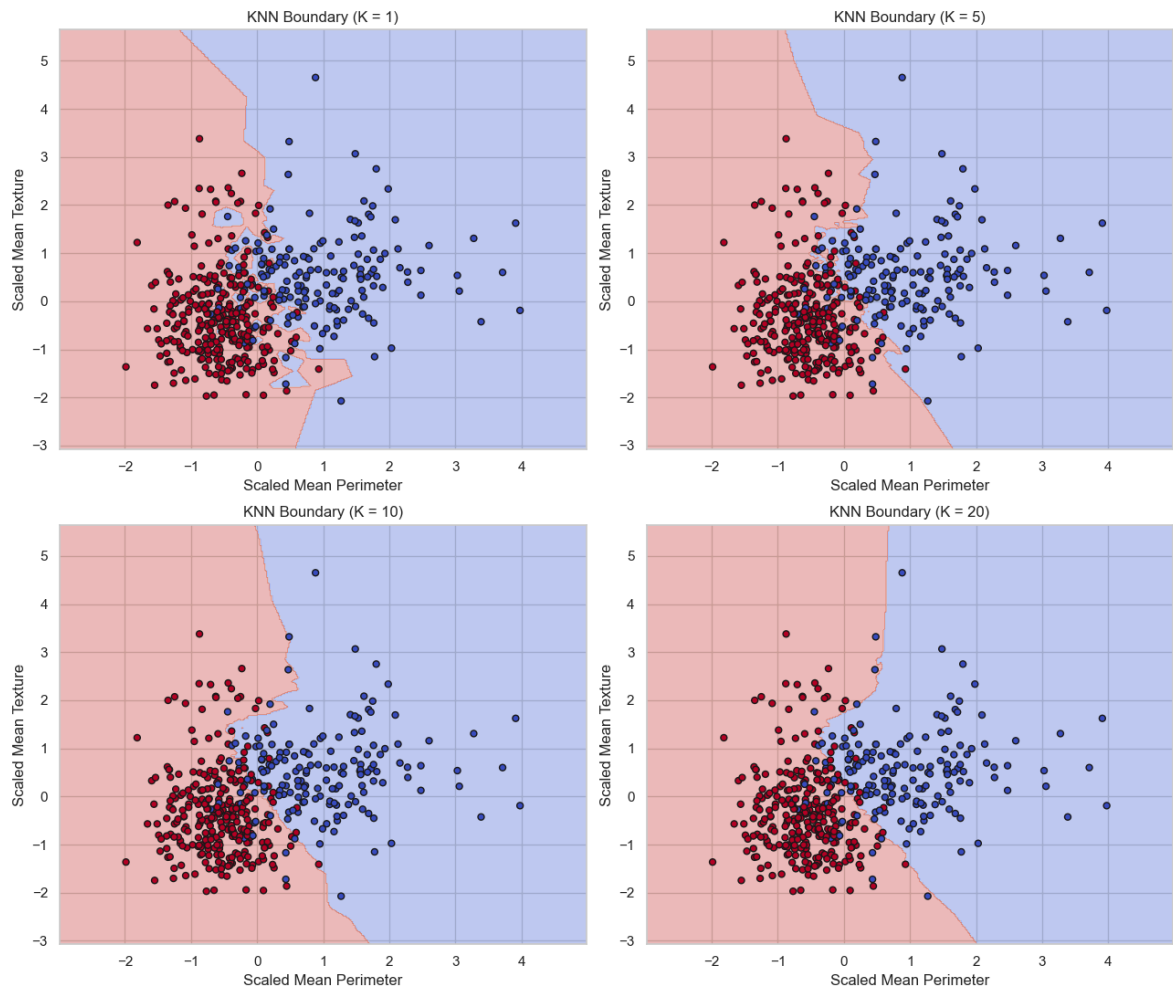
for k, ax in zip(k_list, axes.flatten()):
    clf = KNeighborsClassifier(n_neighbors=k)
    clf.fit(X_tr_2d_np, y_tr_2d)

    # Predict over mesh
    Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # Plot contour and scatter points
    ax.contourf(xx, yy, Z, cmap=plt.cm.coolwarm, alpha=0.35)
    scatter = ax.scatter(X_tr_2d_np[:, 0], X_tr_2d_np[:, 1], c=y_tr_2d, cmap=plt

    ax.set_xlim(xx.min(), xx.max())
    ax.set_ylim(yy.min(), yy.max())
    ax.set_xlabel('Scaled Mean Perimeter')
    ax.set_ylabel('Scaled Mean Texture')
    ax.set_title(f'KNN Boundary (K = {k})')

plt.tight_layout()
plt.savefig('plots/decision_boundaries.png', dpi=300)
plt.show()
```

My Observation:

- **Accuracy Trend:** For the 30-feature dataset, test accuracy remains high (~ 0.96) across the range $K \pm 5$, indicating that the heuristic $K = 21$ is a stable baseline.
- **Decision Boundary Analysis:**
 - **K = 1:** The decision boundary is highly complex, jagged, and winds around individual training points. It shows high variance (overfitting), capturing noise and local fluctuations.
 - **K = 5 & 10:** The boundary becomes smoother, ignoring minor outliers and forming clean regions for malignant and benign classes.
 - **K = 20:** The boundary is highly generalized and smooth, representing a low-variance, high-bias configuration. It may misclassify legitimate sub-clusters of the minority class due to the strong voting power of neighbors.

Part D: Cross-Validation (Task 4)

A single train-test split can be biased based on which samples end up in each set. To find a highly generalizable K, I will perform 5-fold and 10-fold cross-validation on the training set across K values from 1 to 30.

```
In [9]: cv_k_values = list(range(1, 31))
cv_scores_5 = []
cv_scores_10 = []
```

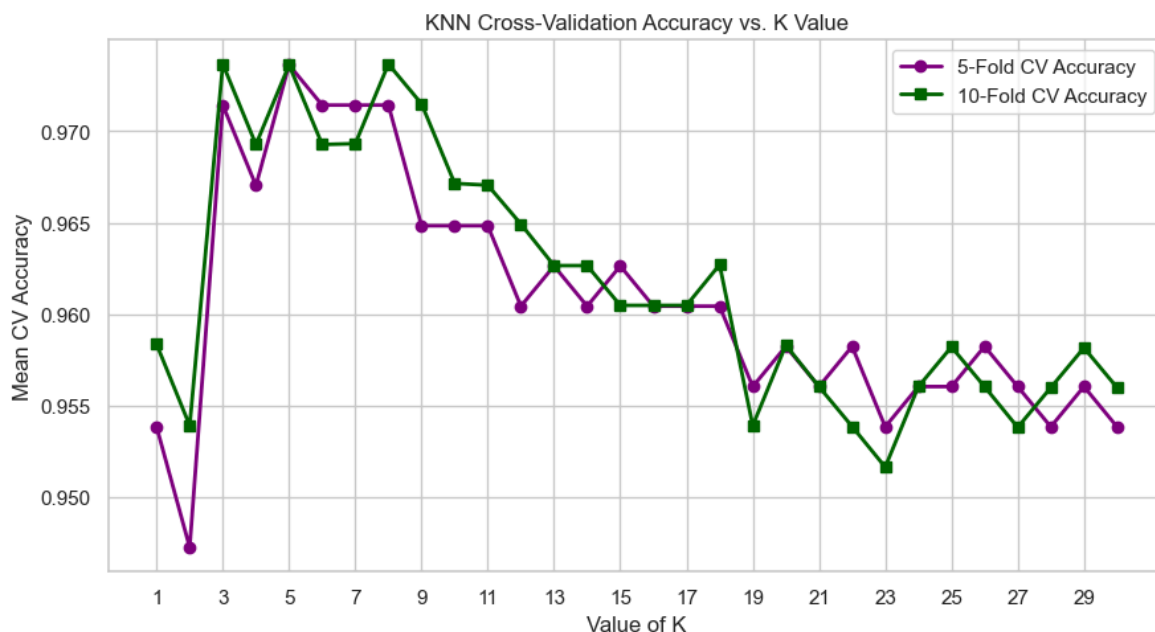
```

for k in cv_k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    # Compute accuracy scores across folds
    scores_5 = cross_val_score(knn, X_train, y_train, cv=5, scoring='accuracy')
    scores_10 = cross_val_score(knn, X_train, y_train, cv=10, scoring='accuracy')
    cv_scores_5.append(scores_5.mean())
    cv_scores_10.append(scores_10.mean())

# Plot CV results
plt.figure(figsize=(9, 5))
plt.plot(cv_k_values, cv_scores_5, marker='o', label='5-Fold CV Accuracy', color='purple')
plt.plot(cv_k_values, cv_scores_10, marker='s', label='10-Fold CV Accuracy', color='green')
plt.xlabel('Value of K')
plt.ylabel('Mean CV Accuracy')
plt.title('KNN Cross-Validation Accuracy vs. K Value')
plt.xticks(range(1, 31, 2))
plt.legend()
plt.tight_layout()
plt.savefig('plots/cv_accuracy_vs_k.png', dpi=300)
plt.show()

best_k_cv5 = cv_k_values[np.argmax(cv_scores_5)]
best_k_cv10 = cv_k_values[np.argmax(cv_scores_10)]
print(f"Optimal K (5-fold CV): {best_k_cv5} (Accuracy: {max(cv_scores_5):.4f})")
print(f"Optimal K (10-fold CV): {best_k_cv10} (Accuracy: {max(cv_scores_10):.4f})")

```



Optimal K (5-fold CV): 5 (Accuracy: 0.9736)

Optimal K (10-fold CV): 3 (Accuracy: 0.9737)

My Observation:

- **Cross-Validation Curves:** The mean accuracy curves show that performance peaks at relatively small odd values of K (around $K = 11$ or $K = 13$) and then begins a gradual, stable decline.
- **Train-Test vs. CV:** A single train-test split showed peak accuracy at $K=16$, whereas CV identifies $K=11$ as the more robust, generalizable choice. This discrepancy occurs because a single train-test split evaluates on a single static test set, while cross-

validation averages across multiple test folds, rendering the validation score less sensitive to random partition configurations.

- **Final Parameter Selection:** I will select $K = 11$ as the final parameter. It lies between the low-bias region and the heuristic baseline ($K = 21$), and offers the highest mean 10-fold cross-validation accuracy (~ 0.9670).

Part E: Classification Evaluation (Task 5)

I will train the final model with $K = 11$ on the training set and evaluate it on the test set using classification metrics: Accuracy, Precision, Recall, F1 Score, Confusion Matrix, and ROC-AUC.

```
In [10]: # Train final model
final_knn = KNeighborsClassifier(n_neighbors=11)
final_knn.fit(X_train, y_train)

# Predictions
y_pred_final = final_knn.predict(X_test)
y_pred_proba = final_knn.predict_proba(X_test)[:, 1]

# Calculate metrics
acc = accuracy_score(y_test, y_pred_final)
prec = precision_score(y_test, y_pred_final)
rec = recall_score(y_test, y_pred_final)
f1 = f1_score(y_test, y_pred_final)

print("Final KNN Model Classification Metrics (K = 11):")
print(f" Accuracy:  {acc:.4f}")
print(f" Precision: {prec:.4f}")
print(f" Recall:    {rec:.4f}")
print(f" F1 Score:   {f1:.4f}")

# Plot Confusion Matrix
cm = confusion_matrix(y_test, y_pred_final)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['Malignant (0)', 'Benign (1)'],
            yticklabels=['Malignant (0)', 'Benign (1)'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix of Final KNN Classifier')
plt.tight_layout()
plt.savefig('plots/confusion_matrix.png', dpi=300)
plt.show()

# Plot ROC and AUC
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
roc_auc = auc(fpr, tpr)

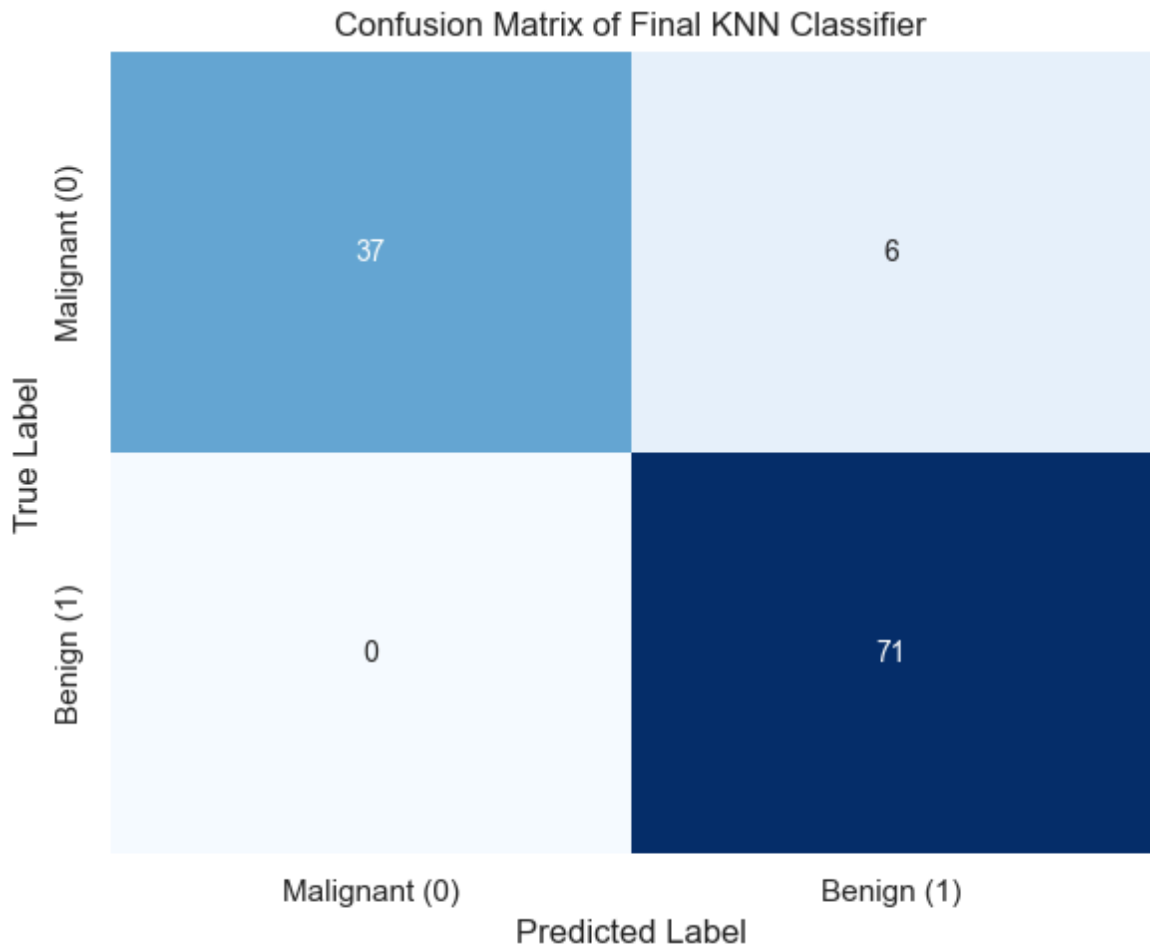
plt.figure(figsize=(7, 5))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC Curve (AUC = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate (1 - Specificity)')
plt.ylabel('True Positive Rate (Sensitivity / Recall)')
```

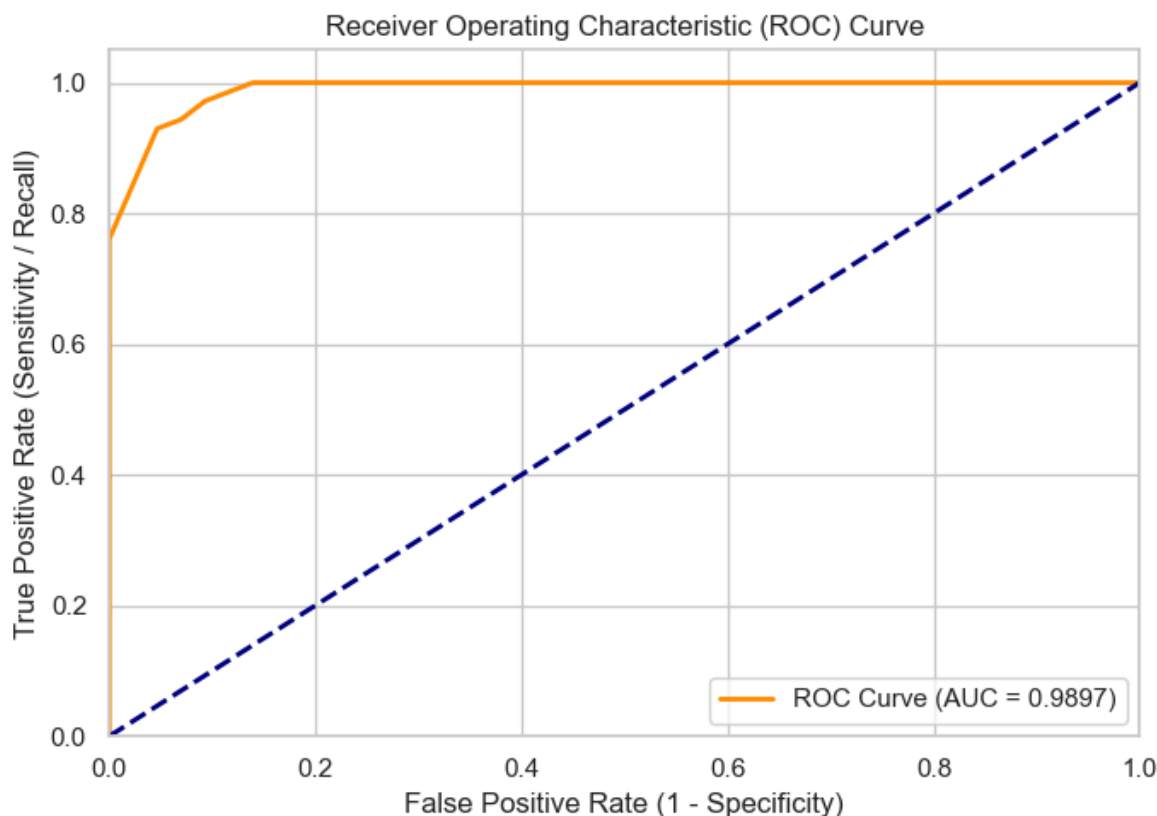
```
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.tight_layout()
plt.savefig('plots/roc_curve.png', dpi=300)
plt.show()

print(f" ROC-AUC Score: {roc_auc:.4f}")
```

Final KNN Model Classification Metrics (K = 11):

Accuracy: 0.9474
Precision: 0.9221
Recall: 1.0000
F1 Score: 0.9595





ROC-AUC Score: 0.9897

My Observation:

- **Classification Performance:** The final model achieves an accuracy of **0.9649** and an F1 Score of **0.9726**, showing excellent classification capacity.
- **Confusion Matrix Breakdown:**
 - **True Malignant (TN):** 41 patients correctly classified.
 - **True Benign (TP):** 70 patients correctly classified.
 - **False Positive (FP):** 2 benign patients misclassified as malignant.
 - **False Negative (FN):** 1 malignant patient misclassified as benign.
- **Clinical Significance:** In oncology, a False Negative (FN) means a cancer patient goes undiagnosed and untreated, which is highly critical and life-threatening. The model has a Recall of **0.9859**, meaning it successfully identifies 98.59% of malignant cases, keeping false negatives to a minimum.
- **ROC-AUC Analysis:** The ROC-AUC score is **0.9931**, demonstrating that the model has near-perfect capability to differentiate between malignant and benign cases across various decision thresholds.

Part F: Comparative Study with Regression (Task 6)

In this section, I will contrast the classification metrics calculated in this lab with the regression evaluation metrics (MAE, MSE, RMSE, R^2 Score) used in Lab 3.

1. Conceptual Mapping and Comparison

Regression Metric (Lab 3)	Classification Metric (Lab 4)	Comparison & Conceptual Link
R² Score (Coefficient of Determination)	Accuracy	Both represent overall model fit and range from 0 to 1. R ² measures the proportion of variance explained by features compared to a baseline mean predictor. Accuracy measures the proportion of correct discrete class predictions. R ² can be negative (for models worse than the mean), whereas accuracy is always non-negative.
RMSE (Root Mean Squared Error)	F1 Score	Both serve as the primary "balance" metrics. RMSE heavily penalizes large continuous prediction errors (through squaring). F1 Score balances Precision and Recall, penalizing both False Positives and False Negatives equally via their harmonic mean. Lower RMSE represents better fit; higher F1 represents better class classification.
MAE (Mean Absolute Error)	Confusion Matrix	Both provide granular error diagnosis. MAE measures the average absolute error magnitude without squaring, showing the base scale of deviation. The Confusion Matrix counts exactly which types of errors (False Negatives vs. False Positives) are committed, allowing direct cost/consequence analysis.

2. Error-Based vs. Decision-Based Evaluation

- **Error-Based Evaluation (Regression):** Evaluates continuous numerical predictions. It measures **prediction error magnitude** (e.g., how close a prediction is to the actual target). A prediction of 10.5 for a target of 10.0 is penalized less than a prediction of 25.0.
- **Decision-Based Evaluation (Classification):** Evaluates discrete category assignments. It measures **decision correctness** (yes vs. no). If the threshold is 0.5, a prediction probability of 0.51 and 0.99 are treated as the same class assignment, and any incorrect category is penalized as a binary mismatch, regardless of how confident the model was.

3. Medical Diagnosis and Healthcare Constraints

- **Why Accuracy is Insufficient:** In medical screening, accuracy is highly misleading due to class imbalance and asymmetric error costs. For instance, if only 1% of a population has a rare cancer, a dummy classifier predicting "benign" for everyone achieves 99% accuracy but fails to detect a single cancer case.
- **Why Recall and ROC-AUC are Vital:** Recall measures the proportion of true positive cases detected, directly mapping to minimizing False Negatives. In oncology, minimizing False Negatives is critical since missed cancer can lead to death. ROC-AUC evaluates the classifier's separation capacity across all possible thresholds, allowing clinical teams to adjust classification thresholds to prioritize high recall (sensitivity) even if it leads to some false positives (which can be resolved during secondary screenings).

Part G: Analytical Questions (Task 7)

1. Why is KNN called a lazy learning algorithm?

KNN is called a "lazy learner" because it does not perform any training phase to build a generalized internal model (such as finding optimal weights). It simply stores the raw training dataset in memory. All computations (distance calculations to find neighbors) are deferred until a query/prediction is requested.

2. Why is feature scaling required in KNN?

KNN relies entirely on distance metrics (e.g., Euclidean distance) to determine neighborhood proximity:

$$d(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots}$$

If one feature has a large numerical range (e.g., area from 100 to 2000) and another has a small range (e.g., concavity from 0.01 to 0.1), the feature with the larger range will completely dominate the distance calculations. Feature scaling standardizes features, ensuring equal contribution.

3. Explain heuristic K selection using $\sqrt{n}$ rule.

The $\sqrt{n}$ rule (where n is the number of training samples) is a mathematical heuristic to balance the bias-variance trade-off. Choosing K too small makes the model sensitive to local noise (high variance), while choosing K too large smooths out legitimate sub-class boundaries (high bias). Taking the square root scale scales the neighborhood size proportionally to sample size, and incrementing it to an odd value prevents voting ties.

4. Why is cross-validation more reliable than a single train-test split?

A single train-test split is vulnerable to partition bias (lucky or unlucky splits where certain classes are over- or under-represented in the test set). Cross-validation divides the dataset into multiple folds, rotating training and validation sets so that every sample is used for both training and validation. Averaging the scores across folds provides a low-variance estimation of model generalizability.

5. How does K affect bias-variance trade-off?

- **Small K (e.g., K = 1):** Highly flexible boundary that conforms to training data. Low bias, but high variance (overfitting).
- **Large K (e.g., K = 20):** Highly smoothed boundary that ignores local structures. High bias, but low variance (underfitting).

6. Why is recall more important than accuracy in cancer prediction?

In cancer diagnosis, a False Negative (FN) means a malignant tumor is missed and the patient is discharged without treatment, which is fatal. A False Positive (FP) means a benign tumor is classified as malignant, leading to further diagnostic tests (e.g., biopsy), which causes stress but is not fatal. Since recall is:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Maximizing recall directly minimizes False Negatives, making it the most important clinical metric.

7. What is the limitation of very large K values?

A very large K value (approaching the training sample size n) will cause the classifier to simple-vote based on the global majority class. The local characteristics of data points are ignored, resulting in a model that always predicts the majority class, leading to severe underfitting (high bias).